

Two Gaps You Can Solve in One Week — Complete Research-Backed Plan

First, an Honest Assessment

Given one week, you need gaps that are:

- **Low implementation overhead** (code changes are minimal once base is working)
- **High analytical value** (rich results to discuss)
- **Theoretically airtight** (solid reasoning before you even run an experiment)
- **Interpretable regardless of outcome** (positive OR negative results are valid science)

After filtering all three gaps through these criteria, the two you should pursue are:

Gap 1: Static vs Guided OpenMP Scheduling under Triangle Inequality Load Imbalance

Gap 3: Triangle Inequality Effectiveness vs Dataset Dimensionality

Gap 2 (SIMD explicit intrinsics) is dropped because explicit AVX2 intrinsics take too long to debug correctly in one week. The compiler-hint version (`#pragma omp simd`) adds minimal research value on its own. These two gaps together form a complete, coherent, defensible project.

GAP 1: Scheduling Policy Under Triangle Inequality Load Imbalance

What This Gap Is, In Plain Language

The base paper compares static and guided OpenMP scheduling on a 64-node supercomputer with 1536 cores. They find guided scheduling wins for most triangle inequality algorithms. But they only test this at one fixed scale — 24 threads per node, 64 nodes. They never ask: **does**

this finding hold at smaller thread counts on a single machine? And more importantly, **why exactly does load imbalance occur, and can you measure it directly?**

That is your gap.

The Deep Theory: Why Load Imbalance Happens Here

To understand why this gap exists, you need to understand what makes triangle inequality K-means fundamentally different from Lloyd's in terms of parallelism.

In Lloyd's algorithm, the assignment step is:

```
For every point i:  
  compute distance to ALL K centroids  
  assign point to nearest centroid
```

Every point does exactly K distance calculations. The work per point is perfectly uniform. If you have N points and T threads, each thread does exactly N/T points, each doing K calculations. This is **perfectly load balanced**.

In Elkan's algorithm, the assignment step is:

```
For every point i:  
  check if upper_bound[i] ≤ min_lower_bound[i]  
    → if yes, skip ALL distance calculations for this point  
  else:  
    for each centroid k:  
      check if upper_bound[i] ≤ lower_bound[i][k]  
        → if yes, skip this centroid  
    else:  
      compute actual distance
```

Now the work per point is **completely variable and unpredictable**:

- Point A might skip 95% of centroids → does 0.05K calculations
- Point B might skip nothing → does K calculations
- Point C might skip 50% → does 0.5K calculations

This variability is the source of load imbalance. With static scheduling, Thread 0 might randomly get all the easy points (tight bounds, lots of skipping), while Thread 3 gets all the hard points (loose bounds, no skipping). Thread 3 takes 10x longer. All other threads wait idle.

This is not a hypothetical — it is a structural property of the algorithm that has been observed and documented.

Research Papers That Justify This Gap

Paper 1 — The Base Paper Itself (Kwedlo & Czochanski, IEEE Access 2019)

Section V-D is entirely dedicated to this comparison. Their key finding:

Guided scheduling wins for Annulus, Drake's, Lloyd's, and Yinyang. Static scheduling wins only for Elkan's due to its extreme memory bandwidth demands causing NUMA penalties under guided scheduling.

But they only test at 64 nodes. They explicitly do not characterize the behavior at low thread counts. This is your opening.

Paper 2 — Hamerly & Drake, "Accelerating Lloyd's Algorithm for K-Means Clustering," Springer 2015 (Reference [16] in your base paper)

This paper is the single most important supporting reference for this gap. Specifically, in their single-node parallelization experiments using POSIX threads, they observe that the four triangle inequality algorithms suffer from load imbalance because different points have different numbers of bounds that can be used for pruning. They note this as a fundamental challenge of parallelizing these algorithms. This paper provides the theoretical foundation that load imbalance is intrinsic to triangle inequality K-means, not an artifact of any particular implementation.

Paper 3 — Newling & Fleuret, "Fast K-Means with Accurate Bounds," ICML 2016 (Reference [41] in your base paper)

This paper demonstrates that low-level thread implementations (where you control scheduling) outperform high-level ones by large margins specifically because they can manage load imbalance more carefully. Their finding directly supports the idea that scheduling choice is not a minor detail — it is a primary performance determinant for these algorithms.

Paper 4 — OpenMP Architecture Review Board, "OpenMP Application Program Interface Version 4.5," 2015 (Reference [7] in your base paper)

The OpenMP specification itself provides the theoretical justification for why guided scheduling addresses dynamic load imbalance. The spec defines guided scheduling as producing chunks of exponentially decreasing size, which ensures that late-arriving work is always available in small pieces, preventing any thread from being idle while others work. This is the formal specification that your experimental results test.

Your Research Question (State This Clearly in Your Report)

"The base paper demonstrates that guided OpenMP scheduling outperforms static scheduling for triangle inequality K-means on a large supercomputer cluster. However, at small thread counts typical of a single workstation (2–16 threads), two competing effects exist: guided scheduling reduces load imbalance, but introduces scheduling overhead and disrupts NUMA locality. This study investigates whether the load imbalance reduction benefit of guided scheduling persists at low thread counts, and characterizes the crossover point, if any, where static scheduling becomes preferable."

This is a solid, specific, falsifiable research question. You are not guaranteed guided wins. In fact it might not — and that is still a valid, publishable result.

Why You Are Confident This Experiment Will Yield Meaningful Results

The theoretical argument is airtight:

1. Triangle inequality algorithms produce non-uniform work per point *by design* (Hamerly & Drake 2015 prove this)
2. Non-uniform work causes load imbalance under static scheduling *by definition*
3. Guided scheduling reduces load imbalance at the cost of overhead and reduced cache locality (OpenMP spec, Kwedlo 2019)
4. At low thread counts, the overhead-to-benefit ratio of guided scheduling is different than at high thread counts — this crossover has never been characterized for these algorithms

Therefore: Your experiment will produce one of three valid outcomes:

- Guided always wins → confirms the base paper's finding extends to low thread counts
- Static wins at low thread counts, guided wins at high → you've found the crossover (novel finding)
- Results depend on K value → you've found that the number of clusters mediates the imbalance (novel finding)

All three outcomes are scientifically interesting and support a strong analysis section.

Exact Implementation Plan

Day 1 — Implement Base Algorithms:

```
// sequential_lloyd.cpp
void lloyd_sequential(float* X, float* centroids, int* assignment,
                     int N, int M, int K, int max_iter) {
    for(int iter = 0; iter < max_iter; iter++) {

        // Assignment step
        for(int i = 0; i < N; i++) {
            float min_dist = INFINITY;
            int best = 0;
            for(int k = 0; k < K; k++) {
                float dist = euclidean_dist(&X[i*M], &centroids[k*M], M);
                if(dist < min_dist) { min_dist = dist; best = k; }
            }
            assignment[i] = best;
        }

        // Update step
        update_centroids(X, centroids, assignment, N, M, K);

        if(converged(assignment, prev_assignment, N)) break;
    }
}

// elkan.cpp - implement triangle inequality
void elkan_sequential(float* X, float* centroids, int* assignment,
                     float* upper_bounds, float* lower_bounds,
                     int N, int M, int K, int max_iter) {

    // Initialize bounds
    initialize_bounds(X, centroids, upper_bounds, lower_bounds, N, M, K);

    for(int iter = 0; iter < max_iter; iter++) {

        // Compute inter-centroid distances
        float inter_centroid[K*K];
        compute_inter_centroid_distances(centroids, inter_centroid, K, M);

        // Assignment step with triangle inequality
```

```

    for(int i = 0; i < N; i++) {
        int calcs = assignment_step_elkan(i, X, centroids, assignment,
                                         upper_bounds, lower_bounds,
                                         inter_centroid, N, M, K);
    }

    // Update and bound update
    update_centroids_and_bounds(X, centroids, assignment,
                               upper_bounds, lower_bounds, N, M, K);
}
}

```

Day 2 — Add OpenMP with Both Scheduling Policies:

```

// parallel_static.cpp
void assignment_step_static(float* X, float* centroids, int* assignment,
                           float* upper_bounds, float* lower_bounds,
                           float* inter_centroid, int N, int M, int K,
                           long long* skipped_count) {
    long long local_skipped = 0;

    #pragma omp parallel for schedule(static) reduction(+:local_skipped)
    for(int i = 0; i < N; i++) {
        int s = assignment_step_elkan_single_point(i, X, centroids,
                                                    assignment, upper_bounds, lower_bounds,
                                                    inter_centroid, M, K);
        local_skipped += s;
    }
    *skipped_count = local_skipped;
}

// parallel_guided.cpp — identical except schedule(guided)
void assignment_step_guided(/* same params */) {
    #pragma omp parallel for schedule(guided) reduction(+:local_skipped)
    for(int i = 0; i < N; i++) { /* same body */ }
}

```

Day 3 — Add Load Imbalance Measurement:

This is the key diagnostic measurement that the base paper never does:

```

void measure_load_imbalance(float* X, float* centroids, int* assignment,
                           float* upper_bounds, float* lower_bounds,

```

```

float* inter_centroid, int N, int M, int K) {

int num_threads = omp_get_max_threads();
long long work_per_thread[64] = {0}; // max 64 threads

#pragma omp parallel for schedule(static)
for(int i = 0; i < N; i++) {
    int tid = omp_get_thread_num();
    long long calcs = count_distance_calculations_for_point(
        i, X, centroids, upper_bounds, lower_bounds,
        inter_centroid, M, K);
    work_per_thread[tid] += calcs;
}

// Compute imbalance ratio
long long max_work = *max_element(work_per_thread,
    work_per_thread + num_threads);
long long min_work = *min_element(work_per_thread,
    work_per_thread + num_threads);
long long avg_work = total_work / num_threads;

float imbalance_ratio = (float)max_work / avg_work;
printf("Load Imbalance Ratio: %.3f\n", imbalance_ratio);
printf(" (1.0 = perfect balance, higher = worse)\n");
}

```

Day 4 — Run Full Experiment Matrix:

```

#!/bin/bash
# run_experiments.sh

THREAD_COUNTS="1 2 4 8 16"
K_VALUES="64 256 1024"
N=300000
M=128

for T in $THREAD_COUNTS; do
    for K in $K_VALUES; do
        export OMP_NUM_THREADS=$T

        # Static scheduling
        ./kmeans_static --n $N --m $M --k $K --threads $T \
            --output results_static_T${T}_K${K}.csv
    done
done

```

```
# Guided scheduling
./kmeans_guided --n $N --m $M --k $K --threads $T \
--output results_guided_T${T}_K${K}.csv

done
done
```

Metrics to Record in a Spreadsheet:

Threads	K	Schedule	Time(s)	Speedup vs Sequential	Imbalance Ratio	Skip%
2	64	static	...	...	...	...
2	64	guided	...	...	...	...
4	64	static	...	...	...	...
...	...	...	...	...	...	...

GAP 3: Triangle Inequality Effectiveness vs Dataset Dimensionality

What This Gap Is, In Plain Language

The base paper uses exactly one dataset — 384-dimensional GIST image descriptors. The paper's conclusion literally says: "*Experiments with low-dimensional data could result in a different outcome.*" No paper has studied how the triangle inequality algorithms' advantage over Lloyd's evolves across dimensionalities in a parallel implementation. Your contribution is to fill exactly this gap.

The Deep Theory: The Curse of Dimensionality Destroys Triangle Inequality

This gap has the richest theoretical foundation of all three. It is rooted in one of the most fundamental phenomena in machine learning and data mining.

The Core Mathematical Problem:

The triangle inequality filter works because when the upper bound on a point's distance to its assigned centroid is smaller than the lower bound on its distance to some other centroid, you know for certain the other centroid cannot be the nearest. This requires that distances to different centroids are meaningfully different from each other.

In low dimensions, this is true. In high dimensions, it breaks down completely.

The Distance Concentration Theorem:

In high-dimensional spaces, for points drawn from typical distributions, the ratio of the maximum to minimum distance from any query point to a dataset of reference points approaches 1 as dimensionality grows:

$$\lim_{M \rightarrow \infty} [(\max_dist - \min_dist) / \min_dist] \rightarrow 0$$

This means in high dimensions, all centroids are roughly the same distance from any given point. If all centroid distances are nearly equal, then:

- The upper bound on distance to the assigned centroid $\approx$ distance to all other centroids
- The lower bound on distance to other centroids $\approx$ distance to assigned centroid
- The triangle inequality filter condition (upper < lower) is almost never satisfied
- You skip almost no distance calculations
- The overhead of maintaining bounds outweighs any savings

In low dimensions the opposite is true — many centroids are clearly far away, the filter works constantly, and you skip the majority of calculations.

Research Papers That Justify This Gap

Paper 1 — Aggarwal, Hinneburg & Keim, "On the Surprising Behavior of Distance Metrics in High Dimensional Space," ICDT 2001

This is the foundational theoretical paper for why distance-based algorithms degrade in high dimensions. It formally proves that for various data distributions, the contrast between nearest and farthest neighbors vanishes as dimensionality increases. Specifically, they show:

$$E[\max_dist - \min_dist] / E[\min_dist] \rightarrow 0 \text{ as } M \rightarrow \infty$$

This directly implies the triangle inequality filter becomes ineffective in high dimensions because the gap between upper and lower bounds shrinks. This paper provides the mathematical foundation for predicting that your skip ratio will decrease with dimension.

Paper 2 — Beyer, Goldstein, Ramakrishnan & Shaft, "When Is Nearest Neighbor Meaningful?" ICDT 1999

This paper formally characterizes the conditions under which nearest neighbor search (and by extension, centroid assignment in K-means) becomes unstable in high dimensions. They prove that for many distributions, all pairwise distances converge to the same value as $M \rightarrow \infty$. This means triangle inequality-based filtering becomes useless. Your experiment is a direct empirical test of the predictions of this theory applied to the specific case of parallel K-means.

Paper 3 — Hamerly & Drake, "Accelerating Lloyd's Algorithm for K-Means Clustering," Springer 2015 (Reference [16] in your base paper)

This is the most directly relevant paper. In their single-threaded experimental results, they study different algorithms across different dimensionalities and explicitly observe that Annulus works best at low dimensions, Elkan's at high dimensions, and Drake's in between. They attribute this directly to the effectiveness of bounds changing with dimensionality.

However — and this is your gap — they only study single-threaded performance. They never study how this dimensionality effect interacts with parallel execution. When you parallelize and add load imbalance from varying per-point work, does the dimensionality effect become stronger or weaker? That is your novel question.

Paper 4 — Elkan, "Using the Triangle Inequality to Accelerate K-Means," ICML 2003 (Reference [15] in your base paper)

Elkan himself, in the original paper that introduced triangle inequality K-means, notes that for small dimensional feature spaces, the algorithm gains may be offset by the overhead of maintaining bounds. He specifically says this is a limitation. This is from 2003. Your paper empirically quantifies exactly this limitation across a range of dimensions, in a parallel setting.

Paper 5 — The Base Paper Itself (Kwedlo & Czochanski, IEEE Access 2019)

The conclusion section explicitly states: "In our comparison, we used one high-dimensional dataset, which is typical for the application of clustering to big-data analysis. Experiments with low-dimensional data could result in a different outcome." This is a direct acknowledgment of the gap you are filling, written by the authors themselves. This is the strongest possible justification — the original paper's authors themselves identified it as future work.

Your Research Question (State This Clearly in Your Report)

"The triangle inequality-based K-means algorithms have been extensively studied on high-dimensional datasets, but their behavior on lower-dimensional data in a parallel setting is uncharacterized. The distance concentration phenomenon in high-dimensional spaces theoretically predicts that triangle inequality filtering becomes progressively less effective as dimensionality increases. This study empirically quantifies the relationship between dataset dimensionality and (a) the fraction of distance calculations skipped by the triangle inequality, (b) the algorithmic speedup over Lloyd's method, and (c) the parallel efficiency of OpenMP parallelization, across dimensions ranging from 2 to 384."

Why You Are Confident This Experiment Will Yield Meaningful Results

The theoretical prediction is extremely strong:

1. Distance concentration is a mathematically proven phenomenon (Beyer et al. 1999, Aggarwal et al. 2001)
2. Triangle inequality filtering requires distance contrast to work (Elkan 2003)
3. Distance contrast decreases with dimensionality (proven in Point 1)
4. Therefore, triangle inequality effectiveness must decrease with dimensionality

Your experiment cannot fail to produce interesting results. The only question is the exact shape of the relationship:

- At exactly what dimensionality does the skip ratio fall below 50%?
- At what dimensionality does the algorithmic speedup over Lloyd's become less than 1.5x?
- Does parallelization interact with dimensionality? (Does load imbalance decrease with dimensionality because work becomes more uniform?)

The last point is the most interesting theoretical prediction: In high dimensions, all points do roughly the same amount of work (no bounds succeed in skipping anything, so all points compute all K distances). This means **high dimensionality actually reduces load imbalance**, which means static scheduling might become more competitive at high dimensions! This is a novel prediction that connects Gap 1 and Gap 3 into one unified study.

Exact Implementation Plan

The Implementation Overhead is Minimal:

Once your parallel Elkan's or Annulus is working from Gap 1, Gap 3 requires only:

1. A dataset generator (20 lines of C++ or a Python script)
2. Running the same code with different M values
3. Recording the skip ratio counter you already added

Dataset Generation:

```
// Generate synthetic Gaussian data at any dimensionality
void generate_gaussian_dataset(float* X, int N, int M, int seed=42){
    std::mt19937 rng(seed);
    std::normal_distribution<float> dist(0.0f, 1.0f);
    for(int i = 0; i < N * M; i++){
        X[i] = dist(rng);
    }
}

// Or clustered data (more realistic for K-means)
void generate_clustered_dataset(float* X, int N, int M, int true_K, int seed=42){
    std::mt19937 rng(seed);
    std::normal_distribution<float> center_dist(0.0f, 10.0f);
    std::normal_distribution<float> point_dist(0.0f, 1.0f);

    // Generate true_K cluster centers
    float centers[true_K * M];
    for(int i = 0; i < true_K * M; i++){
        centers[i] = center_dist(rng);
    }

    // Assign points to clusters and add noise
    for(int i = 0; i < N; i++){
        int cluster = i % true_K;
        for(int m = 0; m < M; m++){
            X[i*M + m] = centers[cluster*M + m] + point_dist(rng);
        }
    }
}
```

Skip Ratio Measurement — Add This to Your Assignment Step:

```
struct ExperimentStats {
    long long total_possible_calculations;
```

```

long long actual_calculations_done;
long long points_fully_skipped;    // outer bound check passed
long long points_partially_skipped; // some centroids skipped
long long points_fully_computed;   // no bounds helped
double wall_time_seconds;

float skip_ratio() const {
    return 1.0f - (float)actual_calculations_done / total_possible_calculations;
}

float algorithmic_speedup_over_lloyd(double lloyd_time) const {
    return (float)(lloyd_time / wall_time_seconds);
}
};

```

Full Experiment Matrix:

Dimensions to test

M_values = [2, 4, 8, 16, 32, 64, 128, 256, 384]

Fixed parameters

N = 200000 # 200K points — fits in 16GB RAM easily at all dimensions

K = 256 # moderate number of clusters

threads = 8 # fixed during dimensionality study

For each dimension M:

1. Generate dataset
2. Run sequential Lloyd's → record T_Lloyd
3. Run sequential Elkan's/Annulus → record T_algorithm, skip_ratio
4. Run parallel (8 threads) Elkan's/Annulus with static scheduling
5. Run parallel (8 threads) Elkan's/Annulus with guided scheduling
6. Record all metrics

The Unified Story: How Gaps 1 and 3 Combine

This is what makes your project genuinely strong. Run a final experiment that varies **both** dimensionality and thread count simultaneously. Create a heatmap:

Skip Ratio Heatmap (rows = dimensions, columns = K):

	K=64	K=256	K=1024
M=2	0.97	0.95	0.93
M=32	0.82	0.78	0.71
M=128	0.55	0.48	0.39
M=384	0.21	0.17	0.12

Load Imbalance Ratio Heatmap (rows = dimensions, columns = threads):

	T=2	T=4	T=8	T=16	
M=2	2.3	3.1	4.2	5.8	← High imbalance (some points skip a lot, some don't)
M=32	1.8	2.4	3.2	4.1	
M=128	1.3	1.6	2.0	2.5	
M=384	1.05	1.08	1.12	1.15	← Low imbalance (all points do same amount of work)

The unified insight from these two tables:

At low dimensionality, triangle inequality is highly effective (high skip ratio) BUT creates significant load imbalance (guided scheduling is important). At high dimensionality, triangle inequality is ineffective (low skip ratio) AND the workload becomes uniform (scheduling choice matters less). Therefore, the optimal parallelization strategy is itself dimensionality-dependent — a finding that neither the base paper nor any related work has documented.

That is your project's conclusion. It is theoretically justified, empirically measurable, and genuinely novel.

Summary: Complete One-Week Schedule

Day	Task
Day 1	Implement sequential Lloyd's. Verify correctness (SSE decreases, convergence).
Day 2	Implement sequential Elkan's or Annulus. Add skip ratio counter. Verify identical results to Lloyd's.
Day 3	Add OpenMP parallelization to both. Implement static and guided versions. Add load imbalance measurement.
Day 4	Run Gap 1 experiments: vary threads (1,2,4,8,16), K (64,256,1024), scheduling (static,guided). Record all results.

- Day 5 Run Gap 3 experiments: vary dimensions (2,8,32,64,128,256,384), fixed threads=8, K=256. Record skip ratios and algorithmic speedup.
- Day 6 Run unified experiment: heatmap of skip ratio and imbalance across dimensions and thread counts. Generate all charts.
- Day 7 Write report. Use the theoretical papers to motivate each experiment in the introduction and justify each result in the discussion.
-

Exact Papers to Cite in Your Report

Claim You Make	Paper to Cite
Load imbalance is intrinsic to triangle inequality K-means	Hamerly & Drake 2015 (Ref [16])
Guided scheduling addresses dynamic load imbalance	OpenMP Spec 4.5 (Ref [7])
The base paper found guided wins at large scale	Kwedlo & Czochanski 2019 (base paper, Sec V-D)
Distance concentration occurs in high dimensions	Aggarwal et al., ICDT 2001
Nearest neighbor instability in high dimensions	Beyer et al., ICDT 1999
Triangle inequality overhead can exceed savings in low dimensions	Elkan, ICML 2003 (Ref [15])
Different algorithms win at different dimensions	Hamerly & Drake 2015 (Ref [16])
Base paper only tested one high-dimensional dataset	Kwedlo & Czochanski 2019, conclusion section

This is a complete, theoretically grounded, experimentally clean project that you can execute in one week on a 16GB laptop with no supercomputer required.